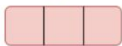
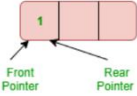
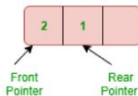
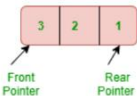


QUEUE

Q8)

Question:

DS	Session	Queue	Question Information	Level 1	Challenge 58
Question description Selvan is very interested in surfing the contents from google. He searches for various coding test on Google. One day he searched about online coding competitions, in the retrieval links, he received many links for coding competition. he chooses first link from the goole suggestion list. first question for the coding competition is LRU cache implementation using queue concepts. Function Description 1. Queue which is implemented using a doubly linked list. The maximum size of the queue will be equal to the total number of frames available (cache size). The most recently used pages will be near front end and least recently pages will be near the rear end. 2. A Hash with page number as key and address of the corresponding queue node as value. Given 3 page frames, so we take size of Queue is 3. Initially, Queue is empty.  Input : 1  Input : 2 (every new input will be front as defined LRU)  Input : 3  Input : 4 (since all the frames are full, we remove a node from the rear of queue, and add the new node to the front of queue.)  Input : 1 (since all the frames are full, we remove a node from the rear of queue, and add the new node to the front of queue.)  Input : 2 (since all the frames are full, we remove a node from the rear of queue, and add the new node to the front of queue.)  Input : 5 (since all the frames are full, we remove a node from the rear of queue, and add the new node to the front of queue.) 					

PROGRAM:

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  struct Node {
5      int data;
6      struct Node *prev;
7      struct Node *next;
8  };
9
10 struct Node *front = NULL;
11 struct Node *rear = NULL;
12 int size = 0;
13 int capacity;
14
15 void moveToFront(struct Node *temp) {
16     if (temp == front)
17         return;
18
19     if (temp == rear)
20         rear = temp->prev;
21
22     if (temp->prev)
23         temp->prev->next = temp->next;
24
25     if (temp->next)
26         temp->next->prev = temp->prev;
27
28     temp->prev = NULL;
29     temp->next = front;
30     front->prev = temp;
31     front = temp;
32 }
33
34 void enqueue(int data) {
35     struct Node *temp = front;
36
37     while (temp != NULL) {
38         if (temp->data == data) {
39             moveToFront(temp);
40             return;
41         }
42         temp = temp->next;
43     }
44
45     temp = (struct Node *)malloc(sizeof(struct Node));
46     temp->data = data;
47     temp->prev = NULL;
48     temp->next = front;
49
50     if (front != NULL)
51         front->prev = temp;
52     else
53         rear = temp;
54
55     front = temp;
56     size++;
57
58     if (size > capacity) {
59         temp = rear;
60         rear = rear->prev;
61         rear->next = NULL;
62         free(temp);
63         size--;
64     }
65 }
```

```
67 void display() {
68     struct Node *temp = front;
69
70     while (temp != NULL) {
71         printf("%d ", temp->data);
72         temp = temp->next;
73     }
74 }
75
76 int main() {
77     int n, m, x;
78
79     scanf("%d %d", &n, &m);
80     capacity = m;
81
82     for (int i = 0; i < n; i++) {
83         scanf("%d", &x);
84         enqueue(x);
85     }
86
87     display();
88
89     return 0;
90 }
```

OUTPUT:

TEST CASE 1:

```
6 4
1 2 3 1 4 5
5 4 1 3
```

TEST CASE 2:

```
10 4
3 1 4 5 9 8 7 6 1 2
2 1 6 7
```